

Data Engineer Interview Prep

Databricks, PySpark, SQL and AWS. 86 questions with interview-ready explanations, tailored to Soudaboina Nagarjuna's resume and the job description.

How to use this document

Each answer gives the **technical answer**, then (where useful) a **SAY IT LIKE THIS** box with a spoken version linked to your own experience, and a **WATCH OUT** box with likely follow-ups or traps. Questions marked **[KEY]** are the highest-probability ones. Your resume claims in the SAY IT boxes are yours to adapt: only state what you can defend in detail.

1. Your position: strengths and gaps

Strong match: Python, PySpark, SQL, AWS (Glue, EMR, S3, Lambda, Redshift), ETL/ELT, data quality, Medallion architecture, migration of 50+ pipelines, Agile, code reviews, mentoring.

Gaps the interviewer will probe:

- **Databricks** is a must-have in the JD. Your resume lists it in skills but none of your projects use it, so expect deep questions. Be honest about what you have used hands-on, then show strong conceptual command: Delta Lake, Auto Loader, Workflows, Unity Catalog, DLT. Always map your Glue/Medallion work to the Databricks equivalent.
- **Scala** is in the JD but not on your resume. Learn the basics and say plainly that Python/PySpark is your primary language.
- **Kafka** is in the JD but not on your resume. Bridge from your SQS + Lambda event-driven project: same decoupled-ingestion idea, Kafka is the replayable, higher-throughput version.

Map your AWS experience to Databricks

What you did on AWS	Databricks equivalent
S3 Bronze/Silver/Gold zones	Delta tables in Bronze/Silver/Gold (Medallion)
Glue job bookmarks	Auto Loader checkpoints / Delta streaming checkpoints
Glue Data Catalog / Hive metastore	Unity Catalog (catalog.schema.table)
MWAA / Airflow DAGs, IBM TWS	Databricks Workflows (or Airflow triggering Databricks jobs)
Custom schema validation + rejection logging	DLT expectations + schema enforcement
Parquet/ORC on S3	Delta (Parquet + transaction log)
YAML configs per env (dev/UAT/prod)	Databricks Asset Bundles / job parameters
Glue DPU tuning	Cluster sizing, autoscaling, Photon, job clusters
SNS alerting	Workflow failure notifications / webhooks
Lake Formation / IAM	Unity Catalog grants, row filters, column masks

If they ask: 'Have you worked on Databricks?'

SAY IT LIKE THIS: My production experience has been on AWS Glue and EMR with a Medallion architecture on S3. I have built the same patterns Databricks formalizes: Bronze/Silver/Gold, incremental loads, schema validation and reconciliation. I understand Delta features like MERGE, time travel, OPTIMIZE, Auto Loader and Unity Catalog, and [state honestly the extent of your hands-on work: labs, POC, certification]. For me the move is mostly a tooling change, not a conceptual one.

WATCH OUT: Do not claim production Databricks experience you do not have. Follow-up questions will expose it quickly. Honest plus well-informed is far stronger.

Your 60-second introduction

SCRIPT: I am a Data Engineer with over 4 years at TCS, working on AWS-based data platforms. My core stack is Python, PySpark, SQL, AWS Glue, EMR and Airflow. Most recently I led the migration of 50+ legacy Informatica pipelines to AWS Glue and MWAA, and built a Medallion data lake on S3, which gave us about 40% better performance and 25% lower cost. I also built the data-quality layer, covering schema validation, rejection logging and reconciliation, that reduced defects by 30%. I have also built an event-driven ingestion pipeline with SQS and Lambda. I am now looking to deepen my work on Databricks and the lakehouse stack, which is why this role interests me, because the concepts I have applied, Medallion, incremental loads and data quality, map directly onto it.

2. Topics to cover (priority order)

Priority	Topic
MUST	Spark internals and tuning (shuffle, skew, joins, partitions, AQE)
MUST	Databricks + Delta Lake (MERGE, time travel, OPTIMIZE, Auto Loader, DLT, Unity Catalog)
MUST	SQL (window functions, joins, CTEs, dedup, optimization)
MUST	Python and PySpark coding
MUST	Your resume projects (Glue migration, Medallion, Lambda pipeline)
HIGH	Data modeling (star schema, SCD), data warehousing
HIGH	Kafka and Spark Structured Streaming basics
HIGH	AWS data services (S3, Glue, EMR, Redshift/Spectrum, Athena, Lambda, EKS, RDS)
HIGH	Data security and privacy (PII, encryption, IAM, masking)
MEDIUM	Hadoop/Hive basics (HDFS, YARN, partitioning vs bucketing)
MEDIUM	Scala basics
MEDIUM	ML pipeline basics (MLflow, feature engineering)
ALWAYS	Behavioral (ownership, estimation, risk, collaboration)

Study plan for the time you have

#	Area	How
1	Databricks and Delta (Q25 to Q35, Q60 to Q65)	Most important. Spend the most time.
2	Spark tuning (Q14 to Q24, Q59)	Practice explaining skew, joins and AQE.
3	SQL and PySpark coding (Q36 to Q50)	Write the code by hand.
4	Resume stories (Q73 to Q78)	Rehearse each aloud in 2 minutes.
5	Kafka, Scala, Hadoop basics (Q51 to Q58)	30 minutes skim.
6	Behavioral (Q79 to Q86)	Prepare 2 to 3 STAR stories.

3. Level 1: Basics (Q1 to Q13)

Q1. What is ETL vs ELT? [KEY]

ETL (Extract, Transform, Load) transforms data in a separate engine *before* loading it into the target. It suits legacy warehouses with limited compute and cases where only clean data may land in the target.

ELT (Extract, Load, Transform) loads raw data first and transforms it inside the target platform (Databricks, Redshift, Snowflake). It uses the target's scalable compute and keeps the raw data available for reprocessing and audit.

- **Where it fits:** modern lakehouses are ELT by nature. Bronze is the 'Load', and Bronze to Silver to Gold is the 'Transform'.
- **Trade-off:** ELT needs stronger governance because raw (possibly sensitive) data lands in the platform.

SAY IT LIKE THIS: In my Glue projects I extracted from RDBMS, APIs and SFG files and landed raw data in Bronze first, then transformed layer by layer. So in practice I have followed the ELT pattern, and I use ETL-style transformations only where a compliance rule required cleansing before landing.

Q2. What is Apache Spark and why is it faster than MapReduce?

Spark is a distributed data processing engine. MapReduce writes intermediate results to disk after every map and reduce stage, but Spark keeps intermediate data in memory and plans the whole job as a DAG, so it can pipeline narrow operations together and avoid needless disk I/O.

- **Other benefits:** unified APIs (SQL, DataFrames, Streaming, MLlib), lazy evaluation with the Catalyst optimizer, and Tungsten for efficient memory and code generation.
- **Architecture:** a Driver (plans and schedules) and Executors (run tasks), coordinated by a cluster manager (YARN, Kubernetes, Databricks).

SAY IT LIKE THIS: I use Spark daily through PySpark on Glue and EMR. The key idea I always keep in mind is that Spark is lazy: nothing runs until an action, which lets Catalyst optimize the entire plan first.

Q3. RDD vs DataFrame vs Dataset?

- **RDD:** low-level, untyped distributed collection. No Catalyst optimization, no schema. Use only when you need fine-grained control.
- **DataFrame:** distributed table with a schema. Optimized by Catalyst and Tungsten. This is the default in PySpark.
- **Dataset:** typed DataFrame (Scala/Java only) with compile-time type safety. Not available in Python.

WATCH OUT: A good follow-up line: 'In Python, DataFrame is effectively Dataset[Row], and I stay on DataFrames and Spark SQL so Catalyst can optimize.'

Q4. Transformations vs actions?

Transformations (select, filter, join, groupBy, withColumn) are lazy: they only add to the logical plan. **Actions** (count, collect, show, write, take) trigger actual execution.

Lazy evaluation lets Spark combine steps, push filters down and prune columns before reading any data.

- **Common trap:** calling `collect()` on a large DataFrame pulls everything to the driver and causes an out-of-memory error. Use `take()`, `limit()` or write to storage instead.

Q5. Narrow vs wide transformations? [KEY]

Narrow (map, filter, select, union): each output partition depends on one input partition, so no data movement and they are pipelined within a stage.

Wide (groupBy, join, distinct, repartition, orderBy): output partitions depend on many input partitions, so data is **shuffled** across the network, which creates a new stage.

Shuffles are the most expensive part of a Spark job (disk I/O, serialization, network). Most tuning is about avoiding or shrinking them.

SAY IT LIKE THIS: When I tuned Glue jobs, my first step was to look for unnecessary wide transformations and replace shuffle joins with broadcast joins wherever one side was small.

Q6. What is Delta Lake? [KEY]

Delta Lake is an open-source storage layer on top of Parquet files in cloud object storage. A transaction log (`_delta_log`) tracks every change, which gives:

- **ACID transactions** (no partial or corrupt writes, safe concurrent readers and writers)
- **Schema enforcement and evolution**
- **Time travel** (query or restore old versions)
- **MERGE / UPDATE / DELETE** on data lake files (plain Parquet cannot do this)
- **Unified batch and streaming** on the same table
- **Performance features:** data skipping, OPTIMIZE compaction, Z-ORDER

Why it matters: plain data lakes suffer from partial failures, no updates or deletes, and no consistency. Delta fixes those, which is why it is the foundation of the lakehouse.

SAY IT LIKE THIS: On S3 with Glue I had to build my own reliability with bookmarks, dead-letter paths and reconciliation because Parquet has no transactions. Delta gives ACID, schema enforcement and MERGE natively, so much of that custom code becomes built-in behaviour.

Q7. What is Databricks? [KEY]

A unified data and AI platform built on Apache Spark and the lakehouse concept. Main components:

- **Compute:** managed clusters (all-purpose, job clusters), SQL Warehouses, serverless, Photon engine
- **Storage format:** Delta Lake
- **Ingestion and pipelines:** Auto Loader, Delta Live Tables, Workflows (jobs)
- **Governance:** Unity Catalog (access control, lineage, audit)
- **ML:** MLflow, Feature Store, Model Serving
- **Development:** notebooks, Repos (Git), Asset Bundles for CI/CD

SAY IT LIKE THIS: I think of it as Spark plus Delta plus governance plus orchestration in one managed platform, which removes the cluster and metadata management overhead I handled manually on EMR.

Q8. What are the Bronze, Silver and Gold layers (Medallion architecture)? [KEY]

- **Bronze:** raw data exactly as received, append-only, with metadata (source, ingestion time, file name). Enables replay and audit.
- **Silver:** cleaned, validated, deduplicated, conformed data. Schema enforced, bad records quarantined, joins across sources.
- **Gold:** business-level aggregates and star-schema models for BI, reporting and ML.

Why it matters: clear separation of concerns, easy reprocessing from Bronze, quality improves progressively, and each layer has a defined consumer.

SAY IT LIKE THIS: On S3 I built this as Bronze, Silver and Gold zones. Bronze kept raw files for replay, Silver applied schema validation, null checks and rejection logging (defects dropped by 30%), and Gold held curated datasets for reporting. That migration improved performance by about 40% and cut cost by 25%.

WATCH OUT: Be ready to explain WHY performance improved: columnar formats, partition pruning, reuse of Silver, and avoiding repeated full-source reads.

Q9. Star schema vs snowflake schema?

Star: one central fact table (measures, foreign keys) surrounded by denormalized dimension tables. Fewer joins, faster queries, simple for BI. Some redundancy in dimensions.

Snowflake: dimensions are normalized into sub-dimensions. Saves storage and reduces redundancy, but needs more joins and is more complex.

For analytics and Gold layers, star schema is the usual choice.

```
fact_sales(sale_id, date_key, product_key, customer_key, qty, amount)
dim_date, dim_product, dim_customer  -- star: one join each
```

Q10. Parquet vs ORC vs Avro vs CSV/JSON?

- **Parquet:** columnar, compressed, supports predicate pushdown and column pruning. Best for analytics on Spark/Databricks. Delta is built on it.
- **ORC:** columnar like Parquet, strong in the Hive ecosystem.
- **Avro:** row-based with embedded schema, strong schema evolution. Best for streaming and Kafka messages.
- **CSV/JSON:** human-readable, but no schema enforcement, larger and slower to parse.

SAY IT LIKE THIS: I wrote Parquet and ORC in my Glue pipelines because columnar storage plus predicate pushdown significantly reduced DPU usage and runtime.

Q11. Primary key vs foreign key vs unique key?

A **primary key** uniquely identifies each row and cannot be null (one per table). A **foreign key** references a key in another table and enforces referential integrity. A **unique key** enforces uniqueness but allows null (depends on the database) and a table can have many.

Note: in lakehouse/warehouse engines like Spark, Delta and Redshift, keys are often informational and not enforced, so integrity checks must be built into the pipeline. You did this in the Unclaimed Asset project (referential integrity checks).

Q12. Types of SQL joins?

INNER (matching rows), LEFT, RIGHT, FULL OUTER, CROSS (cartesian product) and SELF join. Spark adds **LEFT SEMI** (rows in left that have a match, returns only left columns) and **LEFT ANTI** (rows in left with no match). Use LEFT ANTI to find records missing in the target, which is very useful for reconciliation.

```
-- rows in source missing from target
SELECT s.* FROM source s LEFT ANTI JOIN target t ON s.id = t.id
```

Q13. What is data governance?

The policies, roles and tooling that keep data accurate, secure, discoverable and compliant. It covers ownership and stewardship, cataloging and metadata, lineage, quality rules, access control, retention and privacy compliance.

In Databricks, **Unity Catalog** is the governance layer. In AWS, it is Glue Data Catalog plus Lake Formation and IAM.

SAY IT LIKE THIS: In my projects, governance showed up as data quality checks, audit traceability across ingestion layers, secure data access frameworks, and technical documentation such as data dictionaries.

4. Level 2: Intermediate (Q14 to Q58)

Spark (Q14 to Q24)

Q14. repartition vs coalesce? [KEY]

repartition(n) performs a full shuffle and can increase or decrease partitions, producing evenly sized partitions. It can also repartition by column, which helps joins and partitioned writes.

coalesce(n) only decreases partitions by merging existing ones, with no full shuffle. It is cheaper but can leave uneven partition sizes.

- **Rule of thumb:** use repartition to increase parallelism or fix skew, and coalesce to reduce the number of output files before a write.

```
df.repartition(200, 'customer_id')    # shuffle, even, by key
df.coalesce(10).write.parquet(path)  # fewer output files, no shuffle
```

SAY IT LIKE THIS: I used coalesce before writing to S3 to avoid the small-files problem, and repartition by key before large joins to spread the load evenly.

Q15. What is a broadcast join and when do you use it? [KEY]

The small table is copied to every executor, so the large table is joined locally without being shuffled. This removes the most expensive step of a join.

- **Use when:** one side is small enough to fit in executor memory. Default threshold is 10 MB (`spark.sql.autoBroadcastJoinThreshold`) and can be raised.
- **Avoid when:** the 'small' table is actually large. That causes out-of-memory errors on the driver or executors.
- **Alternative for two large tables:** sort-merge join, ideally with bucketing or pre-partitioning on the join key.

```
from pyspark.sql.functions import broadcast
df = large_df.join(broadcast(dim_df), 'id')
```

SAY IT LIKE THIS: In my pipelines I broadcast lookup and reference tables against large transaction data. That was one of the levers behind the runtime and cost improvements.

Q16. cache vs persist?

cache() is shorthand for persist with the default storage level (`MEMORY_AND_DISK` for DataFrames).

persist(level) lets you choose `MEMORY_ONLY`, `MEMORY_AND_DISK`, `DISK_ONLY`, serialized variants, etc.

- **Use when** a DataFrame is reused multiple times, for example in several joins or actions.
- **Always** call `unpersist()` when done.
- **Do not** cache data used only once. It wastes memory and can cause spills.

Q17. What is data skew and how do you fix it? [KEY]

Skew means a few keys hold a disproportionate amount of data, so a few tasks run far longer than the rest and the stage is held up by stragglers. You see it in the Spark UI as a max task time much larger than the median.

Fixes, in order of preference:

- Enable **AQE skew join handling** (`spark.sql.adaptive.skewJoin.enabled`), which splits skewed partitions at runtime.
- **Broadcast** the smaller side if possible.
- **Salting:** add a random suffix to the hot key on the big side and replicate the small side across those suffixes.
- **Isolate hot keys:** process the few heavy keys separately, then union.
- Filter out null keys, which are a very common hidden cause.

```
from pyspark.sql import functions as F
salted_big = big.withColumn('salt', (F.rand()*10).cast('int'))
salted_small = small.crossJoin(spark.range(10).withColumnRenamed('id', 'salt'))
res = salted_big.join(salted_small, ['key', 'salt'])
```

SAY IT LIKE THIS: I diagnose skew from the Spark UI first, comparing task duration and shuffle read size across tasks. Then I choose the least invasive fix, usually AQE or a broadcast, and use salting only if those are not enough.

Q18. What is AQE (Adaptive Query Execution)?

AQE re-optimizes the query plan **at runtime** using real statistics collected after each shuffle stage. Enabled by default in Spark 3.2+ and Databricks.

- **Coalesces small shuffle partitions** (fewer, right-sized tasks)
- **Switches join strategy** (for example sort-merge to broadcast when one side turns out small)
- **Optimizes skew joins** by splitting large partitions

WATCH OUT: Mention that AQE lowers the need to hand-tune `spark.sql.shuffle.partitions`.

Q19. What is the Catalyst optimizer?

Spark SQL's query optimizer. A query goes through four phases: analysis (resolve columns and tables), logical optimization (predicate pushdown, column pruning, constant folding), physical planning (choose join strategies using cost), and code generation (Tungsten whole-stage codegen).

This is why DataFrame/SQL code is faster than RDDs or Python UDFs: Catalyst can see and optimize built-in operations, but not opaque UDFs.

Use `df.explain(True)` to view the plans.

SAY IT LIKE THIS: Predicate pushdown and column pruning on Parquet are Catalyst optimizations, and they were part of how I reduced data scanned and Glue DPU consumption.

Q20. How do you decide the number of shuffle partitions?

Default `spark.sql.shuffle.partitions` is 200. Target roughly 128 to 256 MB per partition after the shuffle. Too few gives spills and OOM, too many gives scheduling overhead and tiny files.

Rule of thumb: $\text{partitions} = \text{shuffle data size} / \text{target size}$, and at least 2 to 3 times the total cores. With AQE on, set a high initial value and let it coalesce.

Q21. UDF vs built-in vs pandas UDF?

Built-in functions are fastest and Catalyst-optimizable. **Python UDFs** are slowest because every row is serialized between the JVM and Python and the optimizer cannot see inside. **Pandas (vectorized) UDFs** use Apache Arrow to process batches and are far faster than row-at-a-time UDFs.

Order of preference: built-in, then SQL expression, then pandas UDF, then plain UDF.

Q22. Explain the small files problem. [KEY]

Thousands of tiny files cause heavy metadata and listing overhead, too many tasks, and slow reads. It is common with streaming writes, over-partitioning (for example partitioning by a high-cardinality column) and frequent incremental loads.

Fixes: coalesce or repartition before write, choose partition columns with low cardinality (date, region), Delta **OPTIMIZE** for compaction, **auto optimize / optimized writes** on Databricks, and Glue file grouping options.

SAY IT LIKE THIS: I addressed this with optimized partitioning and controlling output file counts, which contributed to the 60% improvement in data accessibility on my resume.

Q23. How does Spark execute a job?

An action creates a **job**. The job is split into **stages** at shuffle boundaries. Each stage is split into **tasks**, one per partition. The driver builds the DAG and schedules tasks on executors through the cluster manager. Tasks in a stage run in parallel, and stages run in dependency order.

Q24. How do you handle bad or corrupt records? [KEY]

- Read modes: **PERMISSIVE** (default, bad rows go to `_corrupt_record`), **DROPMALFORMED**, **FAILFAST**.
- **badRecordsPath** in Databricks writes bad records to a location instead of failing.
- Explicit schema validation, null and type checks, and routing failed rows to a **quarantine / dead-letter** path with the reason.
- Alert on rejection percentage above a threshold.

SAY IT LIKE THIS: In my Glue migration, rejected records went to dead-letter S3 paths with rejection logging, and SNS alerts fired on failures. That is how we achieved zero silent failures.

Databricks and Delta Lake (Q25 to Q35)

Q25. How does Delta ensure ACID? [KEY]

Every write is recorded as an atomic JSON commit in `_delta_log`. Data files are written first, then the commit makes them visible in one atomic step, so readers never see partial writes.

- **Atomicity/Consistency:** a commit either fully happens or not at all.
- **Isolation:** optimistic concurrency control. Writers assume no conflict and validate at commit. If two conflict, one retries or fails. Readers use snapshot isolation.
- **Durability:** data and log live in cloud object storage.
- A checkpoint Parquet file is written every 10 commits to speed up log replay.

SAY IT LIKE THIS: Compare it with plain Parquet on S3: a failed job could leave half-written files that downstream jobs read. With Delta, the transaction log prevents that.

Q26. What is time travel and where would you use it?

Query a previous version of a Delta table by version number or timestamp, and roll back with `RESTORE`. Limited by log and file retention (`VACUUM` retention, default 7 days).

Use cases: audit and reproducibility, undoing a bad write, comparing versions for debugging, and reproducing ML training datasets.

```
SELECT * FROM sales VERSION AS OF 12;  
SELECT * FROM sales TIMESTAMP AS OF '2026-09-01';  
RESTORE TABLE sales TO VERSION AS OF 12;  
DESCRIBE HISTORY sales;
```

Q27. OPTIMIZE, Z-ORDER, VACUUM? [KEY]

- **OPTIMIZE:** compacts many small files into larger ones (target around 1 GB) to speed up reads.
- **ZORDER BY (col):** co-locates related values in the same files so queries filtering on that column can skip more files (data skipping). Use on high-cardinality columns commonly used in filters. (Newer Databricks also offers Liquid Clustering, which replaces partitioning and Z-ORDER.)
- **VACUUM:** physically deletes files no longer referenced by the log and older than the retention period (default 7 days). Vacuuming with a short retention breaks time travel and can break long-running readers.

```
OPTIMIZE sales ZORDER BY (customer_id);  
VACUUM sales RETAIN 168 HOURS;
```

SAY IT LIKE THIS: OPTIMIZE and Z-ORDER are the Delta equivalent of what I did manually with partition tuning and file compaction on S3.

Q28. Write a MERGE (upsert) in Delta and explain it. [KEY]

`MERGE` compares a source to a target on a key and applies `UPDATE`, `INSERT` or `DELETE` in one atomic operation. It is the core of incremental loads, CDC and SCD handling.

- **Best practices:** make sure the source has **unique keys** (otherwise the merge fails with multiple-match errors), so dedupe the source first with `ROW_NUMBER`. Add partition or Z-ORDER columns to the `ON` condition so Delta

can prune files. Keep it idempotent.

```
MERGE INTO silver.customers t
USING updates s ON t.id = s.id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

WATCH OUT: PySpark version: `DeltaTable.forName(spark,'silver.customers').alias('t').merge(src.alias('s'),'t.id=s.id').whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()`

Q29. What is Auto Loader? [KEY]

A Databricks feature for incremental file ingestion from cloud storage using the `cloudFiles` source. It discovers new files efficiently via **file notification** (SNS/SQS on AWS) or **directory listing**, tracks progress in a checkpoint (RocksDB), and gives exactly-once processing.

- **Schema inference and evolution** (with a `schemaLocation`) and a rescued data column for unexpected fields.
- Scales to millions of files without re-listing everything.
- Works with `trigger(availableNow=True)` for incremental batch runs on a schedule.

```
(spark.readStream.format('cloudFiles')
  .option('cloudFiles.format','json')
  .option('cloudFiles.schemaLocation','s3://b/schema/')
  .load('s3://b/raw/')
  .writeStream.option('checkpointLocation','s3://b/chk/')
  .trigger(availableNow=True).toTable('bronze.events'))
```

SAY IT LIKE THIS: It is the Databricks equivalent of Glue job bookmarks plus S3 event notifications, which I have used for incremental ingestion, with better scaling and schema evolution.

Q30. What is Delta Live Tables (DLT)? [KEY]

A declarative pipeline framework: you define the tables and transformations, and DLT handles dependency ordering, orchestration, retries, autoscaling, monitoring and lineage.

- **Expectations** are data quality rules: *expect* (log violations), *expect or drop*, *expect or fail*. Metrics are stored in the event log.
- Supports streaming tables and materialized views, and CDC via `APPLY CHANGES INTO`.

```
import dlt
@dlt.table
@dlt.expect_or_drop('valid_id', 'id IS NOT NULL')
def silver_orders():
    return dlt.read_stream('bronze_orders').select('*')
```

SAY IT LIKE THIS: DLT expectations map directly to the data-quality layer I built by hand: null checks, rejection logging and reconciliation.

Q31. What is Unity Catalog?

Databricks' unified governance layer across workspaces. Three-level namespace: **catalog.schema.table**. Also manages volumes, models and functions.

- Centralized access control with `GRANT/REVOKE`, **row filters** and **column masks** for fine-grained security.
- Automatic **lineage** down to column level, audit logs, and data discovery.
- Storage credentials and external locations control access to S3.

SAY IT LIKE THIS: It plays the role Glue Data Catalog plus Lake Formation plus IAM played for me on AWS, but in one place.

Q32. All-purpose vs job clusters?

All-purpose: interactive development, shared, stays up until terminated, higher DBU rate. **Job clusters:** created for a job run and terminated after, lower DBU rate, isolated. Use job clusters (or serverless jobs) in production.

Cost tips: autoscaling, auto-termination, spot instances for workers, cluster pools for fast start, right-sizing memory versus cores.

Q33. Managed vs external tables?

Managed: Databricks manages both metadata and data location. Dropping the table deletes the data. **External:** you specify the location. Dropping the table removes only metadata and the data stays.

Use external tables when data is shared with other systems or must outlive the table definition.

Q34. Schema enforcement vs schema evolution?

Enforcement: Delta rejects writes whose schema does not match the table, which protects data quality. **Evolution:** allows intentional changes, for example `mergeSchema=true` on write or `autoMerge` for MERGE, to add new columns.

WATCH OUT: Say that you keep enforcement ON by default and only evolve schema deliberately, with review.

Q35. How would you deploy Databricks pipelines to production? [KEY]

- Code in Git (Databricks Repos / Git folders) with branch-based workflow and code reviews.
- **Databricks Asset Bundles** (or Terraform) define jobs, clusters and permissions as code, with dev/UAT/prod targets.
- CI/CD in Azure DevOps or Jenkins: lint, unit tests (pytest with a local Spark session), deploy to UAT, integration tests, then approval and prod deploy.
- Secrets in secret scopes, never in code. Monitoring and failure alerts on jobs.

SAY IT LIKE THIS: This is exactly what I did with YAML-configured Glue jobs across dev, UAT and prod through Azure DevOps. The tool changes, and the discipline is the same.

SQL (Q36 to Q44)

Q36. Find the 2nd highest salary.

Two approaches. Prefer `DENSE_RANK` because it generalizes to Nth and handles ties correctly.

```
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rn FROM emp
) t WHERE rn = 2;

-- alternative
SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);
```

Q37. Remove duplicates keeping the latest record. [KEY]

Use `ROW_NUMBER` partitioned by the business key and ordered by the timestamp descending. This is the standard dedup step in Silver layers and before MERGE.

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) rn
  FROM tbl
) WHERE rn = 1;
```

Q38. ROW_NUMBER vs RANK vs DENSE_RANK?

Given values 100, 100, 90: **ROW_NUMBER** gives 1,2,3 (always unique). **RANK** gives 1,1,3 (ties share, gap after). **DENSE_RANK** gives 1,1,2 (ties share, no gap).

Q39. WHERE vs HAVING? DELETE vs TRUNCATE vs DROP?

WHERE filters rows before grouping. **HAVING** filters groups after aggregation.

DELETE removes selected rows (DML, logged, can be rolled back). **TRUNCATE** removes all rows quickly (DDL, minimal logging). **DROP** removes the table structure and data.

Q40. Employees earning more than their manager.

Self join on manager id.

```
SELECT e.name FROM emp e
JOIN emp m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Q41. Running total and month-over-month change.

Window functions: SUM OVER for cumulative totals and LAG for the previous period.

```
SELECT month, revenue,
       SUM(revenue) OVER (ORDER BY month) AS running_total,
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
FROM sales;
```

Q42. How do you optimize a slow SQL query? [KEY]

- Read the **execution plan** first (EXPLAIN) to find scans, large joins and sorts.
- Filter early and select only the needed columns (avoid SELECT *).
- Add indexes on filter and join columns in RDBMS. In a lakehouse use partitioning, Z-ORDER or clustering, and data skipping.
- Replace correlated subqueries with joins or window functions, and avoid functions on filtered columns (they block index use).
- Check join order and join type, and update statistics. In Spark, use broadcast for small tables.

Q43. Indexes: clustered vs non-clustered?

Indexes speed reads but slow writes and use storage. A **clustered** index defines the physical row order (one per table). A **non-clustered** index is a separate structure with pointers to rows (many per table).

Q44. Normalization (1NF, 2NF, 3NF)?

1NF: atomic values, no repeating groups. **2NF**: 1NF plus no partial dependency on part of a composite key. **3NF**: 2NF plus no transitive dependency (non-key columns depend only on the key).

OLTP uses normalized models, while analytics (Gold layer) deliberately denormalizes into star schemas.

Python and PySpark coding (Q45 to Q50)

Q45. List vs tuple? Shallow vs deep copy?

Lists are mutable, tuples immutable and hashable if elements are (usable as dict keys). A shallow copy duplicates the outer object but shares nested objects. A deep copy duplicates everything recursively (copy . deepcopy).

Q46. Generators and decorators?

Generators use yield to produce values lazily, so you can process very large files with constant memory.

Decorators wrap a function to add behavior without changing its code, for example logging, timing, retry or auth.

```
def read_lines(path):
    with open(path) as f:
        for line in f:
            yield line.strip()
```

Q47. Word frequency and duplicates.

Use collections.Counter, which is O(n).

```
from collections import Counter
freq = Counter(text.split())
dups = [w for w, c in freq.items() if c > 1]
```

Q48. Write a retry with exponential backoff.

Useful for flaky APIs and network calls. Mention adding jitter and retrying only on transient errors.

```
import time, random
def retry(fn, tries=3, delay=2):
    for i in range(tries):
        try:
            return fn()
        except Exception:
            if i == tries - 1:
                raise
            time.sleep(delay * 2**i + random.random())
```

SAY IT LIKE THIS: I built Python-based error handling with retries and logging in my log-management system so failures were never silent.

Q49. PySpark: top N per group. [KEY]

Window function with row_number, then filter.

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number, col
w = Window.partitionBy('dept').orderBy(col('salary').desc())
df.withColumn('rn', row_number().over(w)).filter('rn <= 3')
```

Q50. PySpark: null handling, joins, aggregations.

Know these by heart: fillna, dropna, when/otherwise, groupBy with agg, join types, withColumn, and explode.

```
from pyspark.sql import functions as F
df.fillna({'age': 0}).dropna(subset=['id'])
df.withColumn('grade', F.when(F.col('score') >= 90, 'A').otherwise('B'))
df.groupBy('city').agg(F.count('*').alias('n'), F.avg('sales').alias('avg_sales'))
df.join(other, 'id', 'left')
```

Kafka, Hadoop, Hive and Scala (Q51 to Q58)

Q51. Explain Kafka architecture. [KEY]

- **Topic:** a named stream, split into **partitions**. Each partition is an ordered, append-only log and the unit of parallelism.
- **Broker:** a server storing partitions. Each partition has a leader and replicas for fault tolerance.
- **Producer:** writes messages, with the key determining the partition.
- **Consumer group:** consumers share partitions, and each partition is read by exactly one consumer in the group. **Offsets** track progress. Different groups read independently.
- Data is retained for a set time, so it can be **replayed**. Coordination is via ZooKeeper or KRaft.

SAY IT LIKE THIS: I have not run Kafka in production. My event-driven experience is SQS plus Lambda, and I understand Kafka as the replayable, partitioned, higher-throughput equivalent.

Q52. Kafka delivery semantics?

At-most-once (may lose), **at-least-once** (may duplicate, most common), **exactly-once** (idempotent producers plus transactions). With Spark Structured Streaming, checkpointing plus an idempotent or transactional sink such as Delta gives effectively-once results.

Q53. Kafka vs SQS? [KEY]

Kafka: distributed log, messages retained and replayable, multiple independent consumer groups, ordering within a partition, very high throughput, more operational effort (or use MSK/Confluent).

SQS: managed queue, messages deleted after processing, simple, no ops, at-least-once (FIFO queues add ordering and dedup), visibility timeout and DLQ.

Choose Kafka for streaming analytics, event sourcing and replay. Choose SQS for simple decoupling and task queues.

SAY IT LIKE THIS: In my pipeline SQS buffered API payloads and decoupled producers from Lambda processing. Because SQS is at-least-once, I made processing idempotent.

Q54. How do you read Kafka in Spark Structured Streaming?

Use the kafka source, parse the binary value (for example JSON with `from_json`), apply a watermark for late data, and write with a checkpoint location for recovery.

```
raw = (spark.readStream.format('kafka')
      .option('kafka.bootstrap.servers', 'host:9092')
      .option('subscribe', 'orders').load())
parsed = raw.select(F.from_json(F.col('value').cast('string'), schema).alias('d')).select('d.*')
(parsed.withWatermark('event_time', '10 minutes')
  .writeStream.format('delta')
  .option('checkpointLocation', 's3://b/chk/orders').toTable('bronze.orders'))
```

Q55. HDFS and YARN basics?

HDFS: NameNode holds metadata, DataNodes hold blocks (default 128 MB, replication factor 3). **YARN:** ResourceManager allocates cluster resources and NodeManagers run containers. Modern cloud setups replace HDFS with S3 and often YARN with Kubernetes or managed services, separating storage from compute.

Q56. Hive: managed vs external, partitioning vs bucketing? [KEY]

Managed tables: Hive owns the data, and DROP deletes it. **External:** DROP removes only metadata.

Partitioning splits data into directories by column value (use low-cardinality columns such as date). **Bucketing** hashes a column into a fixed number of files (good for high-cardinality columns, joins and sampling).

Note: you migrated the Hive metastore to Glue Data Catalog, so be ready to explain how.

Q57. Scala basics: val vs var, case class, Option.

val is immutable, **var** mutable (prefer val). A **case class** is an immutable data class with pattern matching and automatic equals/toString. **Option** (Some or None) represents optional values and avoids nulls.

```
case class Person(name: String, age: Int)
val df = spark.read.parquet("path")
df.filter($"age" > 30).groupBy("city").count().show()
```

SAY IT LIKE THIS: Python is my primary language and I write PySpark daily. The Spark APIs are the same concepts in Scala, and I am comfortable reading Scala Spark code and learning it quickly.

Q58. Why is Scala used with Spark?

Spark is written in Scala, so new APIs arrive there first. It offers typed Datasets, compile-time checks, and avoids Python serialization overhead for UDFs. PySpark is fine for most ETL because DataFrame operations run in the JVM anyway.

5. Level 3: Advanced and scenario-based (Q59 to Q72)

Q59. A Spark job that ran in 20 minutes now takes 2 hours. How do you debug? [KEY]

Work from the outside in, and say your steps aloud so the interviewer sees a method:

- **1. What changed?** Input volume, schema, code, cluster config, upstream delay, or Spark/runtime version.
- **2. Spark UI:** find the slowest stage, then compare median versus max task time (skew), shuffle read/write size, spill to disk, and GC time.
- **3. Plan:** run explain() and check whether the join strategy changed (for example a broadcast became a sort-merge because the table grew past the threshold).
- **4. Resources:** executor loss, autoscaling limits, contention with other jobs, slow storage listing or small files.
- **5. Fix and verify:** broadcast, salting or AQE, repartition, caching, compaction, or a larger or better-configured cluster. Re-run and compare with the baseline, then add monitoring so it is caught earlier next time.

SAY IT LIKE THIS: I resolved production incidents independently as SME, and my approach is always the same: identify what changed, locate the slow stage in the Spark UI, find the root cause, fix it, and add a guard so it does not recur.

Q60. Design an incremental load pipeline. [KEY]

- **Detect changes:** a watermark column (updated_at or an increasing id), CDC feed, or new files (Auto Loader / Glue bookmarks).
- **Track state:** store the last processed watermark in a control table or checkpoint, and update it only after a successful write.
- **Apply changes:** dedupe the batch, then MERGE into Silver (upsert and delete handling).
- **Make it idempotent:** a rerun of the same batch must not create duplicates.
- **Safety:** lookback window for late data, reconciliation (source versus target counts/sums), alerts, and a backfill path.

```
w = Window.partitionBy('id').orderBy(F.col('updated_at').desc())
latest = src.withColumn('rn', F.row_number().over(w)).filter('rn = 1').drop('rn')
(DeltaTable.forName(spark, 'silver.t').alias('t')
  .merge(latest.alias('s'), 't.id = s.id')
  .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
```

Q61. How do you handle late-arriving data?

Streaming: use event-time windows with watermarks, which define how late data may arrive before state is dropped. **Batch:** reprocess a rolling lookback window (say the last 3 days) and use MERGE so reprocessing is safe. **Dimensions:** insert an 'inferred member' placeholder for a missing dimension key, then update it when the real record arrives. Always log and measure late-arrival rates.

Q62. Implement SCD Type 2 in Delta. [KEY]

SCD2 keeps history: when a tracked attribute changes, the old row is closed and a new row is inserted.

- Columns: business key, attributes, start_date, end_date, is_current.
- Find changed rows by comparing source with the current target row (use a hash of tracked columns for speed).
- In one MERGE: **close** the old row (end_date = today, is_current = false) and **insert** the new version. The usual trick is to union the changed rows with a NULL merge key so they fall into the 'not matched' insert branch.

```
staged = changed.selectExpr('id as merge_key', '*') \
  .unionByName(changed.selectExpr('NULL as merge_key', '*'))
(target.alias('t').merge(staged.alias('s'),
  't.id = s.merge_key AND t.is_current = true')
  .whenMatchedUpdate(condition='t.hash <> s.hash',
    set={'is_current': 'false', 'end_date': 'current_date()'})
  .whenNotMatchedInsertAll().execute())
```

WATCH OUT: If asked for a simpler explanation: SCD1 overwrites, SCD2 keeps history rows, SCD3 keeps a 'previous value' column.

Q63. How do you ensure data quality? [KEY]

Explain it as layers of defence:

- **At ingestion:** schema validation, file and record counts, format checks.
- **In transformation:** null checks, uniqueness, range/domain checks, referential integrity.
- **After load:** reconciliation between source and target (counts, sums, checksums) and audit logging.
- **Handling failures:** quarantine bad records with the reason, do not silently drop, alert on thresholds.
- **Tooling:** DLT expectations, Great Expectations, or custom framework. Track quality metrics over time.

SAY IT LIKE THIS: I built exactly this: schema validation, null checks, rejection logging and automated reconciliation, which reduced defects by 30% and supported 99.9% accuracy. In Unclaimed Asset Reporting I added referential integrity and cross-system reconciliation with full audit traceability.

WATCH OUT: Be ready to explain how the 30% and 99.9% were measured. Say something concrete such as defect tickets before versus after, or mismatched records divided by total.

Q64. How do you design for idempotency and exactly-once? [KEY]

Idempotent means running the same job twice gives the same result. Achieve it with deterministic keys, MERGE instead of blind append, partition overwrite (replaceWhere), dedup on a business key, and checkpoints. Streaming exactly-once = replayable source + checkpoint + idempotent/transactional sink (Delta).

```
df.write.format('delta').mode('overwrite') \
    .option('replaceWhere', "load_date = '2026-09-30'").save(path)
```

Q65. Design an end-to-end pipeline: S3 to Databricks to BI. [KEY]

Walk left to right, then cover the cross-cutting concerns:

- **Source to landing:** databases, APIs and files land in S3 (or via Kafka/SQS).
- **Bronze:** Auto Loader ingests raw data into Delta with metadata columns.
- **Silver:** clean, validate, dedupe, MERGE, quarantine bad rows.
- **Gold:** star schema and aggregates, optimized with OPTIMIZE and Z-ORDER or Liquid Clustering.
- **Serve:** Databricks SQL Warehouse to BI tools (Tableau, Power BI). ML reads from Silver/Gold or the Feature Store.
- **Orchestration:** Databricks Workflows or Airflow. **Governance:** Unity Catalog. **Security:** encryption, IAM, masking.
- **Ops:** CI/CD, monitoring, alerting, cost controls (job clusters, autoscaling, spot).

SAY IT LIKE THIS: I have built this shape on AWS with Glue, S3 and Airflow. On Databricks the components change names, and the architecture is the same.

Q66. How do you secure sensitive data and protect privacy? [KEY]

- **At rest:** S3 SSE-KMS, encrypted volumes. **In transit:** TLS.
- **Access control:** IAM least privilege and roles, Unity Catalog or Lake Formation for table, row and column permissions.
- **PII protection:** classify data, mask or tokenize or hash PII, use dynamic views or column masks so only authorised roles see raw values.
- **Secrets:** AWS Secrets Manager or Databricks secret scopes. Never hard-code credentials.
- **Audit and compliance:** CloudTrail and Databricks audit logs, retention and deletion policies (GDPR/DPDP style right to erasure, which Delta supports via DELETE plus VACUUM).

SAY IT LIKE THIS: My resume mentions secure data access frameworks. Explain concretely what you implemented, such as IAM roles, restricted S3 access and masked datasets.

Q67. How would you migrate 50+ legacy pipelines? [KEY]

- **Assess:** inventory pipelines, dependencies, volumes, SLAs and complexity, then prioritize into waves (quick wins first).
- **Standardize:** build a reusable, modular, config-driven framework so each migration is mostly configuration.
- **Migrate in waves** with parallel runs: old and new side by side.
- **Validate:** reconcile outputs (counts, sums, sample diffs) before cutover.
- **Cut over, monitor, decommission** the old system, and document.
- **Risks:** hidden logic in legacy tools, scheduling dependencies, data type differences, and rollback plans.

SAY IT LIKE THIS: This is my Informatica DEI to AWS Glue story: modular PySpark scripts, YAML configs for dev/UAT/prod, Hive metastore moved to Glue Data Catalog, and TWS schedules refactored into event-driven MWAA DAGs.

Q68. Glue vs EMR vs Databricks: when to use which? [KEY]

- **Glue:** serverless Spark, low ops, integrated with Glue Catalog. Best for moderate ETL and teams that want no cluster management. Less tuning control.
- **EMR:** full control over Spark/Hadoop/Hive, flexible instances and spot, often cheapest at very large scale, but you manage clusters.
- **Databricks:** best developer experience, Delta, Photon, Auto Loader, DLT, Unity Catalog governance and ML in one platform, at a higher DBU cost.

Decision drivers: team skills, governance needs, workload scale, streaming and ML requirements, cost and lock-in.

SAY IT LIKE THIS: I have used Glue and EMR in production, so I can compare them from experience. My view is that Databricks reduces operational overhead and adds governance and performance features I would otherwise build myself.

Q69. Redshift vs Athena vs Spectrum?

Redshift: MPP data warehouse (provisioned or serverless), best for heavy, repeated BI workloads on structured data. **Athena:** serverless SQL on S3, pay per data scanned, great for ad-hoc queries (use Parquet and partitions to cut cost). **Redshift Spectrum:** lets Redshift query S3 external tables, so you can join warehouse and lake data.

Q70. Explain a basic ML pipeline and where a data engineer fits.

Data ingestion, then feature engineering, training, validation, model registry (MLflow), deployment (batch or real-time) and monitoring for drift.

The data engineer builds reliable, versioned, quality-checked feature and training datasets, automates and schedules pipelines, supports a Feature Store, and makes sure the training and serving data match (avoiding training-serving skew).

Databricks touchpoints: MLflow tracking and registry, Feature Store, Delta time travel for reproducible training sets.

WATCH OUT: This is a 'good-to-have' area. A short, accurate answer is enough. Do not overclaim ML experience.

Q71. Batch vs streaming: how do you choose?

Choose by the latency the business really needs. Streaming when minutes or seconds matter (fraud, monitoring). Batch when hourly or daily is fine because it is simpler, cheaper and easier to test. A middle path is incremental batch with `trigger(availableNow=True)`, giving streaming semantics and checkpointing at batch cost.

Q72. How would you reduce cloud cost for pipelines? [KEY]

- Right-size clusters and use autoscaling and auto-termination.
- Use job clusters, spot/preemptible workers, and serverless where suitable.
- Reduce data scanned: partition pruning, predicate pushdown, columnar formats, compaction.
- Avoid recomputation: incremental loads, caching only when reused, reuse Silver layers.
- Monitor with cost dashboards and tags, and schedule heavy jobs off-peak.

SAY IT LIKE THIS: I reduced cost by 25% through the Medallion redesign, columnar formats, pruning and lower DPU use. Have your exact numbers ready.

6. Resume-based questions (Q73 to Q78)

Q73. Walk me through your Informatica to Glue migration. [KEY]

Use **Problem, Approach, Your role, Result** and keep it around two minutes:

- **Problem:** legacy Informatica DEI pipelines were costly, hard to scale and hard to automate.
- **Approach:** rebuilt ingestion in modular PySpark on Glue (extract, transform, schema validation, write Parquet/ORC to S3), with YAML configs to deploy the same code to dev, UAT and prod through Azure DevOps.
- **Optimizations:** partition pruning, predicate pushdown and columnar storage lowered DPU usage.
- **Reliability:** try/except handling, job bookmarks, dead-letter S3 paths for rejected records, SNS alerts.
- **Platform moves:** Hive metastore to Glue Data Catalog, IBM TWS to event-driven MWAA DAGs.
- **Validation and leadership:** reconciliation between old and new outputs, and code reviews enforcing modular standards.
- **Result:** 40% better performance, 25% lower cost, 99.9% accuracy.

WATCH OUT: Likely follow-ups: what was hardest, how did you validate parity, how did you handle Informatica-specific logic, and what would you do differently.

Q74. How did you get the 40% performance gain and 25% cost reduction? [KEY]

Name the specific levers, and be honest about how you measured:

- Partition pruning and predicate pushdown, so less data was read.
- Parquet/ORC columnar storage and compression.
- Broadcast joins and reduced shuffles, caching of reused data.
- Fewer Glue DPUs and shorter runtimes, plus Medallion reuse so data was not re-read from sources.

How measured: before-versus-after job runtime and DPU-hours across the same pipelines. If a number is approximate, say so rather than overstating.

Q75. What does 99.9% data accuracy mean? [KEY]

Define it clearly: the share of records that passed reconciliation between source and target (counts, sums, checksums) with rejected records tracked and explained. Mention the rejection log and audit trail, and that rejected records were reprocessed or reported, not lost.

Q76. Explain your SQS + Lambda event-driven pipeline. [KEY]

Flow: API payload, SQS (buffer and decouple), Lambda (validate and land), S3 landing zone, EFS staging, EMR/PySpark sanitize and optimize, curated datasets for analytics, all scheduled and monitored with IBM TWS.

Be ready for these follow-ups:

- *Lambda 15-minute limit?* Keep Lambda light (validate and land), and push heavy transformation to EMR.
- *Failures?* SQS visibility timeout, retries and a dead-letter queue.
- *Duplicates?* SQS is at-least-once, so make processing idempotent using unique message keys.
- *Throttling and scale?* Reserved concurrency and batch size control.
- *Why EFS?* Shared file staging between processing steps.

Q77. What is the file-locking mechanism in your log-management system? [KEY]

Prepare a specific answer. A solid structure: the **problem** (concurrent jobs writing or reading the same log or file could corrupt it or read partial data), the **solution** (a lock file or flag acquired before write and released in a finally block, with a timeout and stale-lock cleanup), and the **outcome** (no corrupted files, zero silent failures, clear error logging).

WATCH OUT: This is a likely deep-dive because it is unusual. Rehearse how you implemented it and what happens if a process crashes while holding the lock.

Q78. You say you were SME. Describe a production incident you resolved. [KEY]

Use STAR with one real story: **Situation** (which pipeline, business impact), **Task** (your responsibility), **Action** (how you diagnosed: logs, Spark UI, data checks, then the fix), **Result** (recovery time, data integrity) and **Prevention** (alerting, validation or automation you added).

7. Behavioral and process (Q79 to Q86)

Q79. Tell me about yourself. (Almost certainly the first question.) [KEY]

Keep it to about 60 to 90 seconds: who you are, your strongest achievements, and why this role. A ready script is at the start of this document.

Q80. How do you take ownership of a pipeline project from inception to deployment? [KEY]

Requirements and success criteria, then scoping and breaking work into tasks, estimation with buffer, a risk log, design review, incremental delivery, testing and UAT, a deployment runbook, handover and documentation, and post-release monitoring with clear escalation ownership.

SAY IT LIKE THIS: On the migration I owned the design, the modular framework, code reviews, deployment via CI/CD, and post-migration validation with stakeholders.

Q81. How do you give estimates in sprint planning?

Break stories into tasks, account for unknowns and dependencies, use past velocity and similar past work, add a risk buffer, and state assumptions openly. Re-estimate when scope changes and flag risks early rather than at the deadline.

Q82. How do you work with Data Scientists, Architects and Business SMEs?

Clarify requirements and SLAs up front, agree on data contracts and schemas, share a data dictionary and documentation, demo early, and keep a regular feedback loop. Translate business questions into data requirements and technical trade-offs into business impact.

Q83. How do you handle disagreement in code reviews?

Focus on standards and evidence such as benchmarks and readability, stay respectful, propose an alternative with reasons, and fall back on agreed team conventions or a tech lead if it stays unresolved.

Q84. How do you keep up with new tools?

Give a genuine answer: Databricks release notes and docs, engineering blogs, and small proofs of concept. Name one thing you tried recently.

Q85. Why are you looking for a change, and why this role?

Frame it positively: you want deeper work on Databricks and the modern lakehouse stack, broader ownership of pipelines from inception to deployment, and collaboration across regions and teams. The JD matches your PySpark, AWS and Medallion background. Avoid criticizing your current employer.

Q86. Do you have any questions for us?

Always ask two or three:

- What does the Databricks and AWS architecture look like today, and what are the biggest data challenges the team is solving?
- How are data quality and governance handled (Unity Catalog, DLT)?
- What would success look like in the first 90 days?
- How does the team collaborate across regions?

8. Last-minute checklist

- Practice your 60-second intro and the Informatica to Glue story aloud until they are smooth.
- Know your resume numbers cold (500K+ records/day, 40%, 25%, 99.9%, 30%, 50+ pipelines, 60%) and how each was measured.
- If you do not know something, say so and reason from what you do know. Interviewers value honesty over bluffing.
- Think aloud when answering scenario questions. Structure beats memorized facts.
- For every Databricks question, connect it to what you did on AWS.
- Do not claim production Databricks or Kafka experience you do not have.
- Sleep well. Prepare your setup (camera, mic, internet) and keep your resume open.